

WSI Preprocessing Pipeline

User Guide

Francesca Pecorari

August 12, 2026

1 Overview

This guide describes the implementation of the WSI preprocessing pipeline.

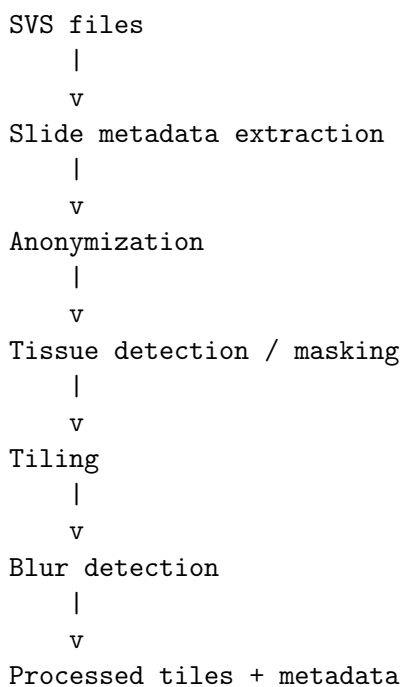
The pipeline processes Whole Slide Images (WSIs) in SVS format and performs a sequence of preprocessing operations, including:

- slide metadata extraction
- slide anonymization
- tissue detection and masking
- WSI tiling
- blur detection
- generation of metadata and processing results

The main input is a collection of SVS files, while the main outputs are processed image tiles and associated metadata.

2 Pipeline Workflow

The overall processing workflow is:



3 Project Structure

The expected project structure is:

```
WSI-Preprocessing/
|-- data1/
|   '-- svsv/
|-- code/
|   |-- .env
|   '-- src/
|       |-- config.py
|       '-- preprocess/
|           |-- anonimization.py
|           |-- profiling.py
|           |-- slide_dataframe.py
|           |-- temp_file.py
|           |-- tiling/
|           |-- blur/
|           |-- masking/
|           '-- pipeline/
|-- logs/
|-- output/
|   |-- pipeline/
|   |-- tiles_ev/
|   '-- slide_metadata/
|-- liver_preprocessing_arm64.sif
|-- run_metadata.sh
'-- test.sh
```

The main directories and files are described below.

data1/svsv/ Contains the input SVS files.

code/ Contains the source code and configuration.

logs/ Contains execution logs.

output/ Contains generated pipeline results.

liver_preprocessing_arm64.sif Apptainer container containing the required software environment.

run_metadata.sh Script used to execute the metadata extraction.

test.sh Script used to execute the pipeline.

4 Requirements

The preprocessing pipeline was implemented in Python 3.10 and containerized using Docker/Apptainer to ensure a reproducible software environment, a .sif file is available for both arm64 and amd64 architectures.

5 Running the Pipeline

Before running the pipeline it is important to modify the `BASE_PATH` and `SIF_PATH` in the `.env` file, all other parameters and folder paths can be found and modified in the `config.py` file.

Make sure you are in the project root:

```
cd /path/to/WSI-Preprocessing
```

The first step is to run:

```
./run_metadata.sh
```

To extract the slide metadata and create the anonymous mapping. Then to run the rest of the pipeline use:

```
./test.sh
```

The pipeline will process the SVS files located in `data1/svs/` and write the results to the configured output directories.

During execution, logs are written to the `logs` folder. The `.sh` file can be modified to match the machine the code is being run on.

6 Pipeline Stages

6.1 Anonymization and Temporary File Management

To protect patient privacy, the WSI preprocessing pipeline anonymizes slide and case identifiers before processing. A persistent mapping between the original case identifiers and randomly generated anonymous identifiers is stored in a JSON file. This mapping is loaded at the beginning of processing and reused throughout the pipeline to ensure that the same case always receives the same anonymous identifier.

For each slide, the original case identifier is replaced while preserving the remaining slide-specific information in the filename. The anonymized slide is then copied to a temporary working directory and used for subsequent processing. Temporary files are removed after the slide has been processed to limit disk usage and prevent unnecessary copies of the original data from being retained.

The anonymization functionality is implemented through the following main functions:

- `load_mapping()` loads the existing case-to-anonymous-ID mapping from the JSON file.
- `get_anonymous_case_id()` assigns a new UUID to a case if it does not already have an anonymous identifier.
- `save_case_mapping()` stores the updated anonymization mapping for future processing.
- `get_anon_slide_id()` generates an anonymized slide filename by replacing the original case identifier while preserving the remaining slide information.
- `anonymized_slide()` creates a temporary anonymized copy of a single slide and removes it after processing.
- `anonymized_batch()` creates temporary anonymized copies for a batch of slides and removes the temporary directory after processing.

6.2 Slide Metadata Extraction

Before image processing, metadata is extracted from each WSI using OpenSlide. The metadata includes the anonymized case identifier, slide name, staining type, number of resolution levels, maximum magnification, and microns per pixel (MPP).

The metadata extraction can be performed in parallel across slides using multiple worker processes. The resulting metadata are stored both as a CSV file and as a Python pickle file for subsequent analysis.

The main functions are:

- `find_svs_files()` recursively searches the input directory for all WSI files.
- `process_slide()` opens an individual WSI with OpenSlide and extracts its relevant metadata.
- `slide_metadata1()` prepares the anonymization mapping and extracts metadata from multiple slides in parallel using a `ProcessPoolExecutor`.

6.3 Tissue Detection and Masking

Tissue detection is carried out at low resolution using the Otsu's thresholding method; the image is converted to grayscale and automatically thresholded to generate a binary mask separating background from tissue. The tiles are then extracted at full resolution.

6.3.1 Main functions

- `make_lowres_mask()` generates the binary low-resolution tissue mask using the configured masking method and applies morphological closing and opening.
- `get_all_tissue_bboxes()` identifies connected tissue regions and returns bounding boxes for regions larger than the minimum area threshold.
- `get_tiling_grid()` generates candidate grid cells from the boundaries of the detected tissue regions.
- `filter_cells_by_mask_fast()` efficiently filters candidate cells according to their tissue coverage and removes regions that are too small for tile extraction.

6.4 Tiling

The tiling step identifies tissue-containing regions and extracts only relevant image tiles.

The overall preprocessing workflow is coordinated by `run_per_slide_anon()`, which processes the input slides sequentially. For each slide, a temporary anonymized copy is created, followed by tissue detection and tiling. The resulting tiles are then passed to the blur detection step. Finally, the processed data are copied from temporary storage to the permanent output directory.

During tiling, a low-resolution version of the WSI is first used to generate a binary tissue mask. Tissue regions are identified and divided into candidate regions, which are filtered according to their tissue coverage. The remaining regions are mapped back to the full-resolution WSI and divided into fixed-size tiles.

The candidate tiles are processed in batches using multiple worker processes. Each tile is extracted using `pyvips` and undergoes a second tissue-content check at full resolution. Tiles with insufficient tissue are discarded, while valid tiles are saved as JPEG images. Metadata containing the tile coordinates, identifiers, and selection status is stored in a CSV file.

6.4.1 Main functions

The main functions involved in this part of the pipeline are:

- `run_per_slide_anon()` orchestrates the processing of all input slides, including anonymization, tiling, blur detection, and output staging.
- `tiling_step()` performs the tiling procedure for an individual slide, including tissue-mask generation and tile extraction.
- `make_lowres_mask()` generates a binary tissue mask from a low-resolution thumbnail.
- `MemorySafeTiler.tile()` identifies tissue regions, generates candidate tile coordinates, distributes the tiles across worker processes, and generates tile metadata.
- `process_tile_batch()` extracts a batch of candidate tiles, evaluates their tissue content, and saves the tiles that pass the tissue threshold.
- `_init_worker()` opens the WSI once for each worker process, avoiding repeated initialization during tile extraction.
- `stage_out()` transfers the generated tiles from temporary storage to permanent storage and merges the blur-detection results into the master CSV file.

6.5 Blur Detection

For detecting blurry tiles both the Tenengrad and Laplacian variance are computed over the tissue regions to assess sharpness, and low tissue regions are divided into smaller blocks, subsequently the Tenengrad score is computed on blocks containing enough tissue to reduce the influence of background on sharpness score. The blur detection is parallelized across multiple worker processes. A timeout mechanism is also used to detect and terminate processing tasks that become unresponsive, allowing the remaining tiles to continue being processed. The classification results and sharpness scores are stored in a CSV file.

6.5.1 Main functions

The main functions involved in the blur detection step are:

- `run_blur_fast()` coordinates the blur detection process, collects the tiles, distributes them across workers, and saves the classification results.
- `process_single_tile()` processes an individual tile by calculating its tissue fraction, selecting the appropriate sharpness method, and classifying it as sharp or blurry.
- `get_tissue_mask()` generates a binary mask identifying tissue pixels.
- `get_tenengrad()` calculates the Tenengrad sharpness score using only tissue pixels.
- `get_laplacian()` calculates the variance of the Laplacian over tissue pixels as a second sharpness measure.
- `get_block_tenengrad()` divides low-tissue tiles into smaller blocks and calculates the Tenengrad score using only blocks with sufficient tissue content.
- `_run_tasks_with_timeout()` manages parallel tile processing and restarts the worker pool if a tile exceeds the defined processing timeout.

7 Output

After successful execution, the output directory should contain the generated results.

For example:

```
output/  
|-- pipeline/  
|   |-- blur_results.csv  
|   |-- slide_001/  
|       |-- tiles/  
|       |-- slide_002/  
|           |-- tiles/  
|-- tiles_ev/  
'-- slide_metadata/
```